AI Receptionist Generation Platform

Production Technical Build Plan

Multi-Tenant AI Voice + Chat Infrastructure for Dental Clinics

Document Classification: Internal — Engineering & Executive

Version: 1.0.0

Date: March 3, 2026

Prepared by: Systems Architecture & Technical Program Office

CONFIDENTIAL — FOR AUTHORIZED RECIPIENTS ONLY

Table of Contents

1. Executive Summary
2. System Architecture Overview
3. Detailed Backend Implementation Plan
   - 3.1 Phase 1 — Core Infrastructure
   - 3.2 Phase 2 — Telephony Integration
   - 3.3 Phase 3 — Provider Abstraction Layer
   - 3.4 Phase 4 — AI Conversation Engine
   - 3.5 Phase 5 — Cost Tracking & Billing Layer
   - 3.6 Phase 6 — Admin Dashboard
4. Database Design Plan
5. Voice Call Runtime Lifecycle
6. Security Architecture
7. Scalability Model
8. Observability & Reliability
9. Cost Optimization Strategy
10. DevOps & Deployment Plan
11. Testing Strategy
12. Implementation Timeline
13. Appendices

---

1. Executive Summary

1.1 What We Are Building

A production-grade, multi-tenant AI receptionist platform that enables dental clinics to deploy always-on, voice-capable AI front-desk agents. Each clinic receives a fully configured AI receptionist — with a dedicated phone number, clinic-specific tone and policies, structured booking logic, and deterministic escalation rules — without managing any AI infrastructure directly.

The platform handles thousands of clinics concurrently, with strict tenant isolation, centralized AI provider orchestration, granular per-call cost attribution, and enterprise-grade reliability.

1.2 Who It Serves

Audience	Value Delivered
Single-location dental clinics	24/7 call answering, automated booking, reduced missed-call revenue loss
Multi-location group practices	Centralized governance with per-location configuration overrides
Dental Service Organizations (DSOs)	Fleet management, SLA enforcement, compliance controls at scale
Platform operators	Operational leverage through shared infrastructure, usage-based monetization

1.3 Core Architectural Decisions

Decision	Rationale
Single Master AI Core with tenant context injection	One reasoning engine serves every clinic — global quality improvements propagate instantly. Tenant behavior is controlled through structured configuration, not separate models.
Dedicated Twilio number per clinic	Deterministic, zero-ambiguity tenant resolution at call ingress. No routing logic ambiguity. Number ownership maps 1:1 to `tenant_id`.
Centralized TTS/STT/LLM provider accounts	Platform-owned accounts enable volume pricing leverage, unified failover, centralized health monitoring, and elimination of per-tenant credential sprawl.
Strict tenant isolation via `tenant_id`	Every database row, cache key, queue message, API call, and analytics event is scoped by `tenant_id`. No cross-tenant data leakage at any layer.
Versioned, immutable tenant configuration	Runtime sessions pin a configuration version at call start. Changes never affect in-flight conversations. Rollback is instant and safe.
Stateless runtime workers	Call-handling workers hold no persistent state. Horizontal scaling is a function of concurrent session count, not tenant count.

1.4 Why Centralized Providers + Isolated Tenants Is Optimal

The centralized-provider/isolated-tenant model is the correct architecture for this class of SaaS platform for five compounding reasons:

1. **Cost efficiency at scale.** Platform-level accounts with aggregate volume across thousands of clinics unlock pricing tiers that individual clinic accounts could never reach. Blended cost per call decreases as tenant count grows.

2. **Operational simplicity.** Managing 3–5 provider account relationships is orders of magnitude simpler than managing thousands of per-tenant credentials with individual billing, rotation, and monitoring.

3. **Unified failover and resilience.** A centralized Provider Selection Engine detects degraded providers across the entire fleet simultaneously and reroutes traffic in milliseconds — no per-tenant failover trees to maintain.
4. **Consistent quality improvements.** Upgrading the Master AI Core or adjusting provider routing improves every tenant simultaneously. No clinic-by-clinic deployment coordination.
5. **Granular attribution without fragmentation.** Per-call cost tracking attributes every STT/TTS/LLM/telephony charge to the originating `tenant_id` and `call_session_id` without requiring separate provider accounts.

---

2. System Architecture Overview

2.1 Five-Layer Architecture Model

The platform is organized into five distinct layers, each with clear responsibilities, ownership boundaries, and interface contracts.

```
┌──────────────────────────────────────────────────────────────┐
│                    LAYER 1 — API LAYER                       │
│  REST/WebSocket Gateway · Auth · Rate Limiting · Routing     │
├──────────────────────────────────────────────────────────────┤
│              LAYER 2 — MULTI-TENANT MIDDLEWARE               │
│  Tenant Resolution · Config Loading · Context Assembly       │
│  Row-Level Security · Tenant-Scoped Caching · Audit Injection │
├──────────────────────────────────────────────────────────────┤
│               LAYER 3 — AI CORE LAYER                        │
│  Master AI Brain · Prompt Orchestrator · Tool Calling Engine │
│  Workflow Planner · Policy Engine · Guardrail System         │
├──────────────────────────────────────────────────────────────┤
│           LAYER 4 — PROVIDER ABSTRACTION LAYER               │
│  Provider Selection Engine · Health Monitor · Failover Router │
│  STT Abstraction · TTS Abstraction · LLM Abstraction         │
│  Cost Metering · Telemetry Collector                         │
├──────────────────────────────────────────────────────────────┤
│             LAYER 5 — INFRASTRUCTURE LAYER                   │
│  PostgreSQL · Redis · Message Queues · Object Storage        │
│  Twilio · External PMS/CRM/Calendar APIs                     │
│  Observability Stack · Secrets Manager · CI/CD               │
└──────────────────────────────────────────────────────────────┘
```

2.2 Layer Responsibilities

Layer 1 — API Layer

Responsibility	Detail
Request ingress	HTTP REST endpoints for dashboard, webhooks, and admin APIs
WebSocket management	Real-time call monitoring and analytics streaming

Authentication	JWT-based auth with short-lived access tokens and refresh rotation
Authorization	Role-based access control (RBAC) with tenant-scoped permissions
Rate limiting	Per-tenant and per-endpoint rate limits with sliding-window counters
Request validation	Schema validation, payload sanitization, content-type enforcement
Webhook reception	Twilio inbound call webhooks, integration callbacks, billing events

Layer 2 — Multi-Tenant Middleware

Responsibility	Detail
Tenant resolution	Resolve `tenant_id` from JWT claims (dashboard), phone number mapping (calls), or API keys (integrations)
Configuration loading	Fetch active `config_version` and hydrate tenant configuration snapshot
Context assembly	Build composite runtime context from tenant config + session state + global instructions
Row-level security	Enforce `tenant_id` predicate on every database query and cache operation
Audit injection	Attach `tenant_id`, `actor_id`, `correlation_id`, and timestamp to every mutating operation
Tenant-scoped caching	Namespace all Redis keys by `tenant_id` with appropriate TTL policies

Layer 3 — AI Core Layer

Responsibility	Detail
Intent understanding	Classify caller intent with confidence scoring
Workflow planning	Select appropriate action path: FAQ, booking, cancellation, escalation
Prompt orchestration	Construct model prompts from global system instructions + tenant context + turn history
Tool calling	Execute booking, calendar lookup, integration queries through typed tool interfaces
Policy enforcement	Validate planned actions against tenant booking rules, escalation policies, and safety constraints
Guardrail system	Prevent hallucination, block prohibited content, enforce emergency protocols
Response generation	Produce caller-facing text within tone, policy, and safety boundaries

Layer 4 — Provider Abstraction Layer

Responsibility	Detail

Provider selection	Score and rank qualified providers by cost, latency, and reliability per request
Health monitoring	Continuous health checks with rolling success/error/latency windows per provider
Failover routing	Automatic rerouting on provider degradation with bounded retry and dead-letter handling
STT abstraction	Unified interface for Deepgram, Google Speech, Whisper, AssemblyAI
TTS abstraction	Unified interface for ElevenLabs, Google TTS, Amazon Polly, PlayHT
LLM abstraction	Unified interface for OpenAI, Anthropic, Google Gemini, open-source models
Cost metering	Capture per-operation usage metrics (tokens, audio seconds, characters) for attribution
Telemetry collection	Emit provider performance data to the scoring algorithm feedback loop

Layer 5 — Infrastructure Layer

Responsibility	Detail
Primary database	PostgreSQL with tenant-partitioned tables, connection pooling, read replicas
Caching layer	Redis for session state, tenant config cache, provider health state, rate limit counters
Message queues	Bull/BullMQ (Redis-backed) for async cost attribution, analytics events, integration callbacks
Object storage	S3-compatible storage for call recordings, exported reports, configuration snapshots
Telephony	Twilio Programmable Voice — inbound/outbound, media streams, recording
External APIs	PMS (Dentrix, OpenDental), CRM, calendar systems via adapter services
Observability	Structured logging, distributed tracing (OpenTelemetry), metrics (Prometheus/Grafana), alerting (PagerDuty)
Secrets management	AWS Secrets Manager / HashiCorp Vault for provider credentials, API keys, encryption keys

2.3 Interaction Diagram — Inbound Call Flow

```
Caller ──────► Twilio ──────► [LAYER 1: Webhook Handler]
                                        │
                                        ▼
                              [LAYER 2: Tenant Resolver]
                              Resolve tenant_id from phone number
                              Load active config_version
                              Assemble runtime context
                                        │
                                        ▼
                              [LAYER 3: Master AI Core]
                           ┌────────────┤
```

```
                    │            │
                Tool Calls    Response Gen
                (booking,     (constrained
                calendar,      by policy
                escalation)    + tone)
                    │            │
                    ▼            ▼
            [LAYER 5:     [LAYER 4: TTS]
            PMS/CRM       Voice synthesis
            adapters]     with tenant voice_id
                                 │
                    ┌────────────┘
                    ▼
            [LAYER 4: STT]  ◄── Caller audio stream
            Transcription
                    │
                    ▼
            [LAYER 5: Cost Attribution]
            Write call_costs record
            Emit analytics events
                    │
                    ▼
            [LAYER 1: Twilio Response]
            TwiML/Media Stream back to caller
```

2.4 Service Topology

Service	Layer	Scaling Unit	State Model
API Gateway	L1	Request rate	Stateless
Auth Service	L1	Auth request rate	Stateless (JWT)
Tenant Config Service	L2	Config read rate	Stateless (cache-backed)
Call Routing Service	L2	Concurrent calls	Stateless
AI Orchestration Service	L3	Concurrent AI sessions	Stateless
Provider Selection Engine	L4	Provider decision rate	Stateless (telemetry cache)
Voice Execution Engine	L4	Concurrent audio streams	Stateless
Integration Service	L5	Integration call rate	Stateless (adapter pool)
Cost Attribution Service	L5	Call completion rate	Queue-driven
Analytics Pipeline	L5	Event ingestion rate	Stream-partitioned

---

3. Detailed Backend Implementation Plan

3.1 Phase 1 — Core Infrastructure

Duration: 3 weeks **Goal:** Establish repository structure, development environment, database foundation, multi-tenant middleware, and base models.

3.1.1 Repository Structure

```
dental-flow/
├── apps/
│   ├── client/                    # Next.js dashboard (existing)
│   └── server/                    # Hono API server (existing)
│       ├── src/
│       │   ├── index.ts           # Server entry point
│       │   ├── config/            # Environment and feature flags
│       │   │   ├── env.ts         # Typed env validation (zod)
│       │   │   └── features.ts    # Feature flag definitions
│       │   ├── db/
│       │   │   ├── index.ts       # Drizzle client
│       │   │   ├── schema.ts      # Drizzle schema definitions
│       │   │   ├── logger.ts      # Query logger
│       │   │   └── migrations/    # Drizzle migration files
│       │   ├── middleware/
│       │   │   ├── auth.ts        # JWT validation middleware
│       │   │   ├── tenant.ts      # Tenant resolution + context injection
│       │   │   ├── rateLimit.ts   # Per-tenant rate limiting
│       │   │   └── audit.ts       # Audit log injection
│       │   ├── modules/
│       │   │   ├── tenants/       # Tenant CRUD, config management
│       │   │   ├── telephony/     # Twilio webhook handlers, number management
│       │   │   ├── calls/         # Call session lifecycle
│       │   │   ├── ai/            # AI orchestration, prompt management
│       │   │   ├── providers/     # Provider abstraction + selection engine
│       │   │   ├── integrations/  # PMS/CRM/calendar adapters
│       │   │   ├── billing/       # Cost attribution, usage tracking
│       │   │   ├── analytics/     # Event processing, dashboard data
│       │   │   └── admin/         # Platform admin operations
│       │   ├── lib/
│       │   │   ├── errors.ts      # Typed error hierarchy
│       │   │   ├── logger.ts      # Structured logger (pino)
│       │   │   ├── queue.ts       # BullMQ queue factory
│       │   │   ├── cache.ts       # Tenant-scoped Redis helpers
│       │   │   └── crypto.ts      # Encryption utilities
│       │   └── types/
│       │       ├── tenant.ts      # Tenant domain types
│       │       ├── call.ts        # Call session types
│       │       ├── provider.ts    # Provider abstraction types
│       │       └── config.ts      # Configuration schema types
│       ├── drizzle.config.ts
│       ├── package.json
│       └── tsconfig.json
├── packages/
```

```
│   └── shared/                    # Shared types, validators, constants
│       ├── src/
│       │   ├── schemas/           # Zod validation schemas
│       │   ├── types/             # Shared TypeScript types
│       │   └── constants/         # Enums, limits, defaults
│       ├── package.json
│       └── tsconfig.json
├── infrastructure/
│   ├── docker/
│   │   ├── Dockerfile.server
│   │   ├── Dockerfile.client
│   │   └── Dockerfile.worker
│   ├── k8s/                       # Kubernetes manifests (future)
│   └── terraform/                 # Infrastructure-as-code (future)
├── docker-compose.yaml
├── pnpm-workspace.yaml
├── package.json
└── turbo.json
```

3.1.2 Environment Configuration

Required environment variables organized by service domain:

Domain	Variables	Validation
Database	`DATABASE_URL`, `DATABASE_POOL_SIZE`, `DATABASE_SSL_MODE`	URL format, numeric range, enum
Redis	`REDIS_URL`, `REDIS_MAX_CONNECTIONS`	URL format, numeric
Auth	`JWT_SECRET`, `JWT_ISSUER`, `JWT_EXPIRY_SECONDS`, `REFRESH_TOKEN_EXPIRY_DAYS`	Non-empty, numeric
Twilio	`TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_WEBHOOK_BASE_URL`	SID format, non-empty, URL format
LLM	`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GOOGLE_AI_API_KEY`	Non-empty
STT	`DEEPGRAM_API_KEY`, `ASSEMBLYAI_API_KEY`	Non-empty
TTS	`ELEVENLABS_API_KEY`, `GOOGLE_TTS_API_KEY`	Non-empty
Storage	`S3_BUCKET`, `S3_REGION`, `S3_ACCESS_KEY`, `S3_SECRET_KEY`	Non-empty
Observability	`LOG_LEVEL`, `OTEL_EXPORTER_ENDPOINT`, `SENTRY_DSN`	Enum, URL, DSN format
Platform	`PLATFORM_ENV`, `PLATFORM_VERSION`, `COST_MARGIN_PERCENT`	Enum, semver, numeric

All environment variables validated at startup using Zod schemas. Application refuses to start with invalid configuration.

3.1.3 Database Setup

Technology: PostgreSQL 16 via Neon (serverless) or RDS **ORM:** Drizzle ORM with typed schema definitions
Migrations: Drizzle Kit with version-controlled migration files

Setup steps:

1. Define all Drizzle schema tables (see Section 4)
2. Generate initial migration from schema diff
3. Create migration runner with idempotency checks
4. Implement connection pool with health monitoring
5. Configure read replica routing for analytics queries
6. Set up automated backup verification

3.1.4 Multi-Tenant Middleware

The tenant middleware is the most critical piece of the platform's isolation model. It must be applied to every tenant-scoped route.

Tenant Resolution Strategy:

Context	Resolution Method	Input
Dashboard API requests	JWT `tenant_id` claim	Authorization header
Twilio webhooks	Phone number → `tenant_id` lookup	`Called` parameter in webhook payload
Integration callbacks	API key → `tenant_id` lookup	`X-API-Key` header
Admin platform routes	Admin JWT with `tenant_id` override	Admin token + explicit tenant param

Middleware execution order:

1. Extract resolution input from request context
2. Resolve `tenant_id` (JWT decode, DB lookup, or API key lookup)
3. Validate tenant status (must be `active`)
4. Load active `config_version` number from `tenant_active_config`
5. Inject `TenantContext` object into request context
6. Set database session variable for row-level security enforcement
7. Initialize tenant-scoped logger with `tenant_id` and `correlation_id`

TenantContext object shape:

```
TenantContext {
  tenantId: UUID
  clinicSlug: string
  status: 'active' | 'suspended' | 'archived'
  activeConfigVersion: number
  resolvedVia: 'jwt' | 'phone_number' | 'api_key' | 'admin_override'
  correlationId: string
  requestedAt: ISO timestamp
}
```

Enforcement rules:

- Any route handler that accesses tenant data MUST receive `TenantContext`
- Database query builders automatically append `WHERE tenant_id = ?` predicates
- Cache keys are prefixed with `tenant:{tenant_id}:`
- Queue messages include `tenant_id` in the job payload
- Failure to resolve tenant returns 401/403 immediately

3.1.5 Base Models and Validation

Zod validation schemas for all domain entities:

- `TenantRegistrySchema` — tenant identity and status
- `ClinicProfileSchema` — clinic details, locations, hours
- `ServiceSchema` — bookable services with constraints
- `BookingRulesSchema` — scheduling policy rules
- `PolicySchema` — escalation, safety, compliance rules
- `VoiceProfileSchema` — voice selection, speed, tone
- `FaqLibrarySchema` — structured FAQ entries
- `IntegrationSchema` — PMS/CRM/calendar configuration
- `CallSessionSchema` — call lifecycle state
- `CallEventSchema` — session event records
- `CallCostSchema` — per-call cost attribution

Each schema includes:

- Type-safe field definitions with enum constraints
- Cross-field validation rules (e.g., `min_notice_hours < max_future_days * 24`)
- Transform functions for normalization (e.g., phone to E.164)
- Custom error messages for user-facing validation feedback

3.2 Phase 2 — Telephony Integration

Duration: 2 weeks **Goal:** Establish Twilio webhook handling, phone number → tenant resolution, call session management, and telephony failover.

3.2.1 Twilio Webhook Design

Webhook endpoints to implement:

Endpoint	Method	Trigger	Response Format
`/webhooks/twilio/voice/incoming`	POST	New inbound call	TwiML
`/webhooks/twilio/voice/status`	POST	Call status change	200 OK
`/webhooks/twilio/voice/recording`	POST	Recording complete	200 OK
`/webhooks/twilio/voice/transcription`	POST	Live transcription event	200 OK
`/webhooks/twilio/voice/gather`	POST	DTMF or speech input	TwiML
`/webhooks/twilio/voice/stream`	WebSocket	Bi-directional media stream	Binary frames

Webhook security:

- Validate `X-Twilio-Signature` header on every request using Twilio auth token
- Reject requests with invalid or missing signatures (HTTP 403)
- Implement request age validation (reject requests older than 5 minutes)
- IP allowlist for Twilio webhook IPs as defense-in-depth

Incoming call webhook handler sequence:

1. Validate Twilio signature
2. Extract `Called` (phone number), `CallSid`, `From`, `CallStatus`
3. Invoke tenant resolution middleware
4. Create `call_sessions` record with status `started`
5. Load tenant active configuration snapshot
6. Return initial TwiML response (greeting or media stream connect)

3.2.2 Phone Number → Tenant Resolution

Primary resolution path:

1. Normalize incoming `Called` number to E.164 format
2. Query `twilio_numbers` table: `WHERE phone_number_e164 = ? AND status = 'active'`
3. Return `tenant_id` from matching record
4. Cache result in Redis with 5-minute TTL: `phone_mapping:{e164}`

Resolution failures:

Failure Mode	Detection	Action
No matching number	NULL result	Return safe fallback TwiML (apology message), emit `CRITICAL` alert
Tenant is suspended	Status check after resolution	Return suspension message TwiML, log event
Tenant is archived	Status check after resolution	Return number-not-in-service TwiML, log event
Database unavailable	Query timeout/exception	Return generic fallback TwiML, emit infrastructure alert

Phone number management operations:

- Provision new Twilio number and associate with `tenant_id`
- Release number association (returns number to pool or releases from Twilio)
- Transfer number between tenants (admin operation with audit trail)
- Primary/secondary number designation per tenant

3.2.3 Call Session Management

Call session lifecycle states:

```
started → in_progress → completed
                      → escalated
                      → failed
```

State transition rules:

From	To	Trigger	Side Effects
`started`	`in_progress`	First AI turn initiated	Emit `call.started` event
`in_progress`	`completed`	Caller hangs up or AI completes	Persist summary, emit `call.completed`, trigger cost attribution

`in_progress`	`escalated`	Human transfer initiated	Persist escalation payload, emit `call.escalated`
`in_progress`	`failed`	Unrecoverable error	Persist error context, emit `call.failed`, trigger incident
`started`	`failed`	Resolution or config load failure	Emit error event

Session data managed in Redis during active call:

- Turn history (last N turns for context window)
- Current intent and confidence
- Actions taken this session
- Escalation state flags
- Provider selections for this call

Session persistence to PostgreSQL:

- On every significant state transition
- At call completion (final write with all accumulated data)
- Critical actions (booking confirmed, escalation triggered) are written immediately

3.2.4 Telephony Failover Handling

Scenario	Detection	Failover Action
Twilio API degradation	Elevated error rate from status callbacks	Queue incoming calls for callback, activate voicemail capture
Media stream disconnection	WebSocket close without call completion	Attempt reconnect (1 retry), then graceful termination with callback offer
TTS/STT provider failure during call	Provider abstraction layer error	Switch to fallback provider mid-call (see Phase 3 failover)
AI Core timeout	Response exceeds 5-second threshold	Play hold message, retry once, escalate to human if second timeout

---

3.3 Phase 3 — Provider Abstraction Layer

Duration: 3 weeks **Goal:** Build unified interfaces for STT, TTS, and LLM providers with dynamic selection, health monitoring, and automatic failover.

3.3.1 Provider Interface Design

Every provider type (STT, TTS, LLM) is accessed through a unified interface contract. Concrete implementations are registered at startup and selected dynamically per request.

Core interface contracts:

STT Provider Interface:

- `transcribe(audio: AudioStream, config: STTConfig): Promise<TranscriptionResult>`

- `streamTranscribe(audio: AudioStream, config: STTConfig): AsyncIterable<TranscriptionChunk>`
- `healthCheck(): Promise<HealthStatus>`
- `getCapabilities(): ProviderCapabilities`

TTS Provider Interface:

- `synthesize(text: string, config: TTSConfig): Promise<AudioBuffer>`
- `streamSynthesize(text: string, config: TTSConfig): AsyncIterable<AudioChunk>`
- `listVoices(filter: VoiceFilter): Promise<Voice[]>`
- `healthCheck(): Promise<HealthStatus>`
- `getCapabilities(): ProviderCapabilities`

LLM Provider Interface:

- `complete(messages: Message[], config: LLMConfig): Promise<CompletionResult>`
- `streamComplete(messages: Message[], config: LLMConfig): AsyncIterable<CompletionChunk>`
- `healthCheck(): Promise<HealthStatus>`
- `getCapabilities(): ProviderCapabilities`

Provider configuration shape:

```
ProviderConfig {
  providerId: string           // e.g., "deepgram", "elevenlabs", "openai"
  providerType: "stt" | "tts" | "llm"
  displayName: string
  enabled: boolean
  priority: number             // Base priority (lower = preferred)
  costPerUnit: number          // Cost per token/second/character
  costUnit: string             // "token" | "second" | "character"
  maxConcurrency: number       // Platform-level concurrency limit
  timeoutMs: number            // Per-request timeout
  retryConfig: RetryConfig
  capabilities: {
    languages: string[]
    features: string[]
    maxInputSize: number
    streamingSupported: boolean
  }
}
```

3.3.2 STT Abstraction

Supported providers (initial):

Provider	Use Case	Strengths	Cost Model
Deepgram	Primary real-time STT	Low latency streaming, dental vocabulary	Per audio second
Google Speech-to-Text	Fallback STT	High accuracy, broad language support	Per 15-second increment

AssemblyAI	Backup STT	Good accuracy, speaker diarization	Per audio second
OpenAI Whisper	Batch/offline transcription	Highest accuracy, no streaming	Per audio second

STT configuration per request:

- Language code (default: `en-US`)
- Encoding format (mulaw, linear16, opus)
- Sample rate (8000 Hz for telephony)
- Interim results enabled (for streaming)
- Punctuation and profanity filtering
- Medical/dental vocabulary boost (where supported)

STT-specific metrics captured:

- Audio duration (seconds)
- Transcription latency (ms)
- Word confidence score
- Provider used
- Whether interim or final result

3.3.3 TTS Abstraction

Supported providers (initial):

Provider	Use Case	Strengths	Cost Model
ElevenLabs	Primary TTS	Natural voice quality, voice cloning	Per character
Google Cloud TTS	Fallback TTS	WaveNet quality, consistent, low cost	Per character
Amazon Polly	Budget fallback	Low cost neural voices	Per character
PlayHT	Alternative primary	Good quality, fast	Per character

TTS tenant configuration applied per request:

- `voice_id` (from tenant `voice_profile`)
- `speaking_speed` multiplier
- `tone` parameter mapping to provider-specific SSML
- `pronunciation_hints` dictionary for dental terms
- Audio output format (mulaw 8000Hz for telephony)

TTS-specific metrics captured:

- Text character count
- Audio output duration (seconds)
- Synthesis latency (ms)
- Provider used
- Voice ID used

3.3.4 LLM Abstraction

Supported providers (initial):

Provider	Models	Use Case	Cost Model
OpenAI	GPT-4o, GPT-4o-mini	Primary reasoning + tool calling	Per input/output token
Anthropic	Claude 3.5 Sonnet, Claude 3 Haiku	High-quality reasoning fallback	Per input/output token
Google	Gemini 1.5 Pro, Gemini 1.5 Flash	Cost-efficient fallback	Per input/output token

LLM request configuration:

- Model ID (selected by Provider Selection Engine)
- Temperature (low for deterministic booking/policy; configurable for FAQ)
- Max output tokens
- Tool definitions (booking, calendar, escalation, FAQ lookup)
- System prompt (global instructions + tenant context)
- Conversation history (windowed)
- Structured output mode (where required)

LLM-specific metrics captured:

- Input token count
- Output token count
- Total token count
- Model used
- Tool calls invoked
- Completion latency (time-to-first-token and total)
- Finish reason (stop, tool_use, length, error)

3.3.5 Health Monitoring

Per-provider health state model:

```
HealthState {
  providerId: string
  providerType: "stt" | "tts" | "llm"
  status: "healthy" | "degraded" | "failing" | "disabled"
  lastCheckedAt: timestamp
  metrics: {
    successRate: number          // Rolling 5-minute window
    p50LatencyMs: number         // Rolling 5-minute window
    p95LatencyMs: number         // Rolling 5-minute window
    errorRate: number            // Rolling 5-minute window
    timeoutRate: number          // Rolling 5-minute window
    consecutiveFailures: number
  }
  circuitBreaker: {
    state: "closed" | "open" | "half_open"
    openedAt: timestamp | null
    halfOpenAt: timestamp | null
    failureThreshold: number     // Failures before opening
    recoveryTimeout: number      // Seconds before half-open
```

```
  }
}
```

Health monitoring cycle:

- Active health checks every 30 seconds per provider (synthetic request)
- Passive health updates on every real request (success/failure/latency)
- Health state stored in Redis with 60-second TTL (auto-expire on monitor failure)
- Circuit breaker transitions emitted as observability events

Status transition rules:

Current	Condition	Next
healthy	Error rate > 5% OR p95 latency > 2x baseline	degraded
degraded	Error rate > 20% OR 5+ consecutive failures	failing
failing	Circuit breaker opens	disabled (temporary)
disabled	Circuit breaker half-open + successful probe	degraded
degraded	Error rate < 2% AND latency normalized	healthy

3.3.6 Failover Strategy

Failover is triggered when:

- Primary provider returns error or timeout
- Circuit breaker is open for primary provider
- Provider health status is `failing` or `disabled`

Failover execution:

1. Provider Selection Engine detects primary failure
2. Exclude failed provider from candidate pool
3. Re-score remaining qualified providers
4. Select next-best provider
5. Retry request with selected fallback provider
6. If fallback also fails, attempt one more provider (max 3 total attempts)
7. If all providers exhausted, trigger graceful degradation path

Graceful degradation paths by modality:

Modality	Degradation	User Experience
STT	All STT providers failing	"I'm having trouble hearing you. Can you please call back in a moment, or press 1 and we'll call you back."
TTS	All TTS providers failing	Fall back to pre-recorded generic prompts, then offer callback
LLM	All LLM providers failing	Transfer directly to front desk with "connecting you to our staff"

3.3.7 Scoring Algorithm

The Provider Selection Engine computes a composite score for each qualified provider per request.

Scoring formula:

```
score = (w_cost × cost_score) + (w_latency × latency_score) + (w_reliability × reliability_score)
```

Default weights:

- `w_cost` = 0.40 (cost is the dominant factor for voice AI workloads)
- `w_latency` = 0.35 (latency directly impacts call experience)
- `w_reliability` = 0.25 (reliability is table-stakes — unhealthy providers are pre-filtered)

Individual score calculations:

Factor	Calculation	Range
`cost_score`	`1 - (provider_cost / max_cost_in_pool)`	0.0–1.0
`latency_score`	`1 - (provider_p50_latency / max_acceptable_latency)` clamped to [0, 1]	0.0–1.0
`reliability_score`	`provider_success_rate` from rolling window	0.0–1.0

Qualification gates (pre-scoring filter):

1. Provider is enabled in platform configuration
2. Provider supports requested capability (language, voice, model, streaming)
3. Provider health status is `healthy` or `degraded` (not `failing` / `disabled`)
4. Provider success rate ≥ 95% (configurable threshold)
5. Provider p95 latency ≤ SLA maximum for workload type
6. Provider is not at concurrency limit

Selection output:

```
ProviderSelection {
  selectedProviderId: string
  providerType: "stt" | "tts" | "llm"
  score: number
  scoreBreakdown: { cost: number, latency: number, reliability: number }
  qualifiedAlternatives: string[]     // Ordered fallback list
  selectionReason: string             // Human-readable
  tenantId: UUID                      // For attribution
  correlationId: string               // For tracing
}
```

3.4 Phase 4 — AI Conversation Engine

Duration: 4 weeks **Goal:** Implement the Master AI Core, prompt orchestration, tool calling, booking workflow, and guardrail system.

3.4.1 Master AI Core Design

The Master AI Core is the central reasoning engine. It is:

- **Stateless** — all state is injected via the runtime context envelope
- **Shared** — one deployment serves all tenants
- **Deterministic in policy** — booking/escalation/safety decisions follow rules, not model creativity
- **Flexible in dialogue** — conversational tone and phrasing adapt to tenant configuration

Core responsibilities per turn:

1. Receive runtime context envelope (tenant config + session state + turn input)
2. Classify caller intent with confidence score
3. Select workflow (FAQ resolution, booking, cancellation, escalation, general conversation)
4. Execute required tool calls within tenant scope
5. Apply policy validation to planned response
6. Generate caller-facing text within tone and safety constraints
7. Return structured turn result (response text, actions taken, updated session state)

Runtime context envelope structure:

```
RuntimeContext {
  // Global
  systemInstructions: string         // Master AI Core system prompt
  coreVersion: string                // AI Core version

  // Tenant (from active config)
  tenantId: UUID
  configVersion: number
  clinicProfile: ClinicProfile
  services: Service[]
  bookingRules: BookingRules
  policies: PolicySet
  voiceProfile: VoiceProfile
  faqLibrary: FaqEntry[]
  integrationCapabilities: IntegrationCapability[]

  // Session
  callSessionId: UUID
  turnHistory: Turn[]
  currentTurnInput: string           // STT transcription
  sessionMetadata: {
    callerPhone: string | null
    callStartedAt: timestamp
    turnsCompleted: number
    actionsCompleted: string[]
    escalationState: EscalationState
  }
}
```

3.4.2 Prompt Orchestration

System prompt composition (ordered priority):

Section	Source	Purpose

1. Core identity	Hardcoded global	"You are the AI receptionist for {clinic_name}..."
2. Safety and compliance	Hardcoded global	Emergency protocols, PHI handling, prohibited actions
3. Clinic profile	Tenant config	Clinic name, hours, locations, timezone
4. Service catalog	Tenant config	Bookable services, durations, eligibility
5. Booking rules	Tenant config	Scheduling policies, constraints, limits
6. Policies	Tenant config	Escalation triggers, sensitive topics, disclaimers
7. Tone instructions	Tenant config	Voice personality, verbosity, greeting style
8. FAQ library	Tenant config	Structured Q&A pairs for common questions
9. Available tools	Platform config	Tool definitions for booking, calendar, escalation
10. Turn context	Session state	Recent conversation history, actions taken

Prompt size management:

- Token budget: 6,000 tokens for system + context (leaves room for conversation + response)
- FAQ entries are ranked by relevance and only top-N are included
- Turn history is windowed to last 10 turns
- Long service catalogs are summarized if exceeding token budget
- Dedicated prompt testing suite validates token usage across tenant configurations

3.4.3 Tool Calling System

Registered tools:

Tool Name	Trigger	Input	Output	Side Effects
`lookup_available_slots`	Caller wants to book	Service code, date range, location	Available time slots	None (read-only)
`create_appointment`	Caller confirms booking	Service, datetime, patient info, location	Booking confirmation	PMS write, event emit
`cancel_appointment`	Caller wants to cancel	Appointment ID or patient+date	Cancellation status	PMS write, event emit
`reschedule_appointment`	Caller wants to reschedule	Old appointment ref, new datetime	Reschedule confirmation	PMS write, event emit
`lookup_faq`	Caller asks common question	Question text, category	Matched FAQ answer + confidence	None (read-only)
`escalate_to_human`	Policy trigger or caller request	Reason, urgency, context summary	Transfer status	Call state update, event emit

`request_callback`	After-hours or fallback	Caller phone, reason, priority	Callback queue confirmation	Queue write, notification
`get_clinic_hours`	Caller asks about hours	Location (optional)	Operating hours	None (read-only)

Tool execution rules:

- All tool calls are tenant-scoped (tool receives `tenant_id` from context)
- Write operations require policy validation before execution
- Tool calls have individual timeouts (3 seconds for reads, 5 seconds for writes)
- Failed tool calls return typed error objects — AI must handle gracefully
- Tool calls are logged as `call_events` with full input/output for audit

3.4.4 Booking Workflow Logic

Complete booking flow (happy path):

1. **Intent detection:** Caller expresses desire to book
2. **Service identification:** AI identifies requested service from catalog
3. **Eligibility check:** Validate patient type, service constraints, booking rules
4. **Slot query:** Call `lookup_available_slots` through integration service
5. **Offer presentation:** Present top 3 available slots in natural language
6. **Caller selection:** Caller chooses preferred slot
7. **Detail collection:** Gather required patient details (name, phone, new/returning)
8. **Policy validation:** Verify booking against all tenant rules (notice period, future limit, double-booking policy)
9. **Confirmation:** Call `create_appointment` through integration service
10. **Verification:** Confirm appointment details back to caller
11. **Event emission:** Emit `booking.confirmed` event with full details

Booking failure handling:

Failure	Detection	Recovery
No available slots	Empty slot query result	Offer alternative dates, waitlist, or callback
Integration timeout	Tool call exceeds 5s	Inform caller, offer to callback with booking, escalate
Slot conflict (race condition)	Write failure from PMS	Re-query slots, offer alternative, log conflict
Policy violation	Guardrail rejection	Explain constraint to caller, offer alternatives within policy
Patient info validation failure	Schema validation error	Re-prompt for corrected information

3.4.5 Guardrails and Validation

Guardrail layers (applied in order):

Layer	Scope	Enforcement

Input guardrails	Transcribed caller text	Detect and flag emergency language, PII patterns, abuse
Intent guardrails	Classified intent	Reject low-confidence intents for critical actions (booking, cancellation)
Policy guardrails	Planned action	Validate against tenant booking rules, escalation policies, safety constraints
Output guardrails	AI-generated response	Filter prohibited phrases, enforce required disclaimers, validate tone compliance
Integration guardrails	External API calls	Validate data formats, enforce idempotency keys, timeout enforcement

Emergency detection and response:

- Pattern matching for emergency keywords (dental emergency, severe pain, bleeding, swelling, trauma)
- Immediate action: deliver emergency disclaimer from tenant config
- Escalation: attempt warm transfer to clinic or emergency routing
- Logging: emit `call.emergency_detected` event with full context

Hallucination prevention:

- AI responses about services, hours, and policies must derive from tenant configuration data
- Booking confirmations require successful tool call confirmation (never confirm without PMS write success)
- "I don't know" responses are preferred over fabricated information
- FAQ answers below confidence threshold trigger "Let me connect you with our staff" fallback

---

3.5 Phase 5 — Cost Tracking & Billing Layer

Duration: 2 weeks **Goal:** Implement granular per-call cost attribution, tenant usage tracking, margin analysis, and billing data pipeline.

3.5.1 Call Cost Records

`call_costs` table (one record per completed call):

Captures the aggregate cost of all providers used during a single call session, attributed to the originating tenant.

Field	Source	Calculation
`stt_provider`	Provider Selection Engine	Provider ID used for transcription
`tts_provider`	Provider Selection Engine	Provider ID used for synthesis
`llm_provider`	Provider Selection Engine	Provider ID used for reasoning
`stt_cost_usd`	STT metering	`audio_seconds × provider_rate_per_second`
`tts_cost_usd`	TTS metering	`characters × provider_rate_per_character`

`llm_cost_usd`	LLM metering	`(input_tokens × input_rate) + (output_tokens × output_rate)`
`telephony_cost_usd`	Twilio usage API	`call_duration_minutes × twilio_rate_per_minute`
`total_cost_usd`	Computed	Sum of all cost components

3.5.2 Call Cost Line Items

`call_cost_line_items` table (multiple records per call):

Captures individual provider operations within a call for detailed attribution and debugging.

Column	Type	Description
`id`	UUID	PK
`tenant_id`	UUID	Partition key
`call_session_id`	UUID	FK to `call_sessions`
`call_cost_id`	UUID	FK to `call_costs`
`provider_id`	text	Provider used
`provider_type`	enum	`stt`, `tts`, `llm`, `telephony`
`operation`	text	e.g., `transcribe`, `synthesize`, `complete`, `call_minute`
`input_units`	numeric	Quantity consumed (seconds, characters, tokens)
`unit_type`	text	`seconds`, `characters`, `input_tokens`, `output_tokens`, `minutes`
`unit_cost_usd`	numeric	Rate per unit at time of operation
`total_cost_usd`	numeric	`input_units × unit_cost_usd`
`latency_ms`	integer	Operation latency
`metadata`	JSONB	Model name, voice ID, additional context
`created_at`	timestamptz	Operation timestamp

Indexes:

- `(tenant_id, call_session_id, created_at)`
- `(tenant_id, provider_type, created_at desc)`

3.5.3 Per-Tenant Attribution

Cost attribution pipeline:

1. During call: each provider operation emits a cost event to an in-memory buffer
2. At call completion: buffer is flushed to `call_cost_line_items` via queue
3. Aggregator job computes `call_costs` summary record from line items
4. Daily aggregation job computes tenant-level daily/weekly/monthly cost summaries

Tenant usage dashboard data points:

- Total cost (today, this week, this month)
- Cost breakdown by provider type (STT, TTS, LLM, telephony)
- Average cost per call
- Cost trend over time
- Top cost drivers (longest calls, most expensive AI reasoning)

3.5.4 Margin Tracking Logic

Platform margin model:

- Platform applies configurable margin on top of provider costs
- `COST_MARGIN_PERCENT` environment variable (default: 30%)
- Displayed separately: `tenant_cost = provider_cost × (1 + margin_percent)`

Margin tracking data points:

Metric	Calculation
Raw provider cost per call	Sum of `call_cost_line_items.total_cost_usd`
Tenant billed cost per call	`raw_cost × (1 + margin)`
Platform margin per call	`billed_cost - raw_cost`
Platform margin per tenant (monthly)	Sum of per-call margins
Blended margin across all tenants	Total margin / total billed cost

3.5.5 Usage Analytics

Aggregate tables/views for analytics:

View	Grain	Key Dimensions
`daily_tenant_usage`	Day × Tenant	Calls, minutes, cost, bookings
`daily_provider_usage`	Day × Provider	Operations, cost, latency p50/p95, error rate
`monthly_tenant_billing`	Month × Tenant	Total cost, margin, cost breakdown
`monthly_platform_economics`	Month × Platform	Revenue, COGS, margin, growth

Usage alerting triggers:

- Tenant daily cost exceeds 2× rolling 7-day average → alert tenant admin
- Tenant monthly cost projected to exceed plan limit → proactive notification
- Single call cost exceeds $5.00 → anomaly flag for review
- Provider cost per unit increases >10% from baseline → platform ops alert

---

3.6 Phase 6 — Admin Dashboard

Duration: 4 weeks **Goal:** Build the clinic-facing dashboard for onboarding, voice configuration, knowledge base management, analytics, and access control.

Technology: Next.js 15 + Tailwind CSS + shadcn/ui (existing client app)

3.6.1 Clinic Onboarding Flow

Multi-step onboarding wizard (stepper component):

Step	Title	Configuration Domain	Required Fields
1	Clinic Identity	`clinic_profile`	Name, legal entity, phone, email, timezone
2	Locations & Hours	`clinic_profile.locations`	Address, operating hours per location
3	Services	`services`	Service names, categories, durations, eligibility
4	Booking Rules	`booking_rules`	Notice period, future limit, cancellation policy, double-booking
5	Policies	`policies`	Escalation conditions, emergency disclaimer, sensitive topics
6	Voice & Tone	`voice_profile`	Voice selection, speed, tone, greeting style
7	Integrations	`integrations`	PMS provider, calendar, notification channel
8	Review & Publish	`tenant_config_versions`	Validation summary, publish trigger

Onboarding UX requirements:

- Progress persistence (can leave and resume)
- Inline validation with real-time field-level feedback
- Step completion indicators (complete, in-progress, blocked)
- Readiness scorecard on review step
- "Continue to AI Chat" button after form completion for refinement

3.6.2 Voice Configuration

Voice configuration UI:

- Voice preview player (play sample audio for each available voice)
- Speed slider (0.8×–1.2×)
- Tone selector (calm, friendly, professional, urgent)
- Pronunciation hints editor (add dental-specific pronunciation overrides)
- Test call button (trigger a test call to admin phone with current config)
- A/B voice comparison tool (play same text in two different voices side by side)

3.6.3 Knowledge Base Management

FAQ management interface:

- Add/edit/delete FAQ entries
- Category-based organization (insurance, hours, procedures, billing, preparation, other)
- Question variant editor (add multiple phrasing variants per FAQ)
- Canonical answer rich text editor
- Confidence threshold slider per FAQ
- Bulk import/export (CSV)
- Search and filter across FAQ library

Service catalog management:

- CRUD for bookable services
- Inline constraint editing (duration, eligibility, approval requirements)
- Drag-and-drop category assignment
- Active/inactive toggle with dependency warnings

3.6.4 Analytics Dashboard

Dashboard pages:

Page	Key Metrics	Chart Types
Overview	Total calls, booking rate, escalation rate, avg response time	KPI cards + area chart trend
Calls	Call volume by hour/day, duration distribution, outcome breakdown	Bar chart + pie chart
Bookings	Bookings made, conversion funnel, popular services, slot utilization	Funnel chart + heatmap
AI Performance	Intent accuracy, resolution rate, avg turns per call, hallucination rate	Line charts + score gauges
Costs	Total spend, cost per call, cost breakdown by provider, trend	Stacked bar + line trend
Quality	Caller satisfaction (post-call), escalation accuracy, false escalation rate	Score cards + trend

Data freshness:

- Real-time call monitoring (WebSocket for active calls)
- Near-real-time analytics (5-minute aggregation lag)
- Daily summary reports (generated at midnight tenant timezone)

3.6.5 Role-Based Access Control

Roles:

Role	Permissions
Owner	Full access: configuration, billing, user management, analytics
Admin	Configuration, analytics, knowledge base, voice settings (no billing)
Manager	Analytics view, call logs, knowledge base edit
Viewer	Analytics view and call logs only (read-only)
Platform Admin	Cross-tenant access, provider management, platform operations

RBAC implementation:

- Roles stored in `tenant_users` table with `role` column
- Permission matrix checked in middleware before route handler execution
- UI components conditionally rendered based on user role

- API endpoints enforce permissions server-side (never client-only)
- Audit log captures all permission-relevant actions with actor identity

---

4. Database Design Plan

4.1 Tenant-Scoped Tables

All tenant-owned tables include `tenant_id` as a required column with index. Every query against these tables MUST include a `tenant_id` predicate.

Tenant configuration tables:

Table	Primary Key	Partition Key	Unique Constraint
`tenant_registry`	`id (UUID)`	—	`(tenant_id)`, `(clinic_slug)`
`twilio_numbers`	`id (UUID)`	`tenant_id`	`(phone_number_e164)`
`clinic_profile`	`id (UUID)`	`tenant_id`	`(tenant_id, config_version)`
`services`	`id (UUID)`	`tenant_id`	`(tenant_id, config_version, service_code)`
`booking_rules`	`id (UUID)`	`tenant_id`	`(tenant_id, config_version)`
`policies`	`id (UUID)`	`tenant_id`	`(tenant_id, config_version)`
`voice_profile`	`id (UUID)`	`tenant_id`	`(tenant_id, config_version)`
`faq_library`	`id (UUID)`	`tenant_id`	`(tenant_id, config_version, faq_key)`
`integrations`	`id (UUID)`	`tenant_id`	`(tenant_id, config_version, integration_type, provider)`
`tenant_config_versions`	`id (UUID)`	`tenant_id`	`(tenant_id, version_number)`
`tenant_active_config`	`tenant_id (UUID)`	—	PK is tenant_id
`tenant_users`	`id (UUID)`	`tenant_id`	`(tenant_id, user_id)`

Runtime and analytics tables:

Table	Primary Key	Partition Key	High-Volume
`call_sessions`	`id (UUID)`	`tenant_id`	Yes
`call_events`	`id (UUID)`	`tenant_id`	Yes
`call_costs`	`id (UUID)`	`tenant_id`	Yes
`call_cost_line_items`	`id (UUID)`	`tenant_id`	Yes
`call_transcripts`	`id (UUID)`	`tenant_id`	Yes

`audit_log`	`id (UUID)`	`tenant_id`	Yes

User and auth tables:

Table	Primary Key	Notes
`users`	`id (UUID)`	Platform user accounts
`tenant_users`	`id (UUID)`	User-to-tenant membership with role
`sessions`	`id (UUID)`	Auth sessions (refresh tokens)

4.2 Global Provider Tables

These tables are not tenant-scoped. They are managed by platform operations.

Table	Primary Key	Purpose
`provider_registry`	`id (UUID)`	STT/TTS/LLM provider definitions and configuration
`provider_health_log`	`id (UUID)`	Historical health check results
`provider_pricing`	`id (UUID)`	Current and historical cost rates per provider per operation
`platform_config`	`key (text)`	Platform-wide feature flags and operational parameters

`provider_registry` columns:

Column	Type	Description
`id`	UUID	PK
`provider_id`	text	Unique identifier (e.g., `deepgram`, `elevenlabs`, `openai`)
`provider_type`	enum	`stt`, `tts`, `llm`
`display_name`	text	Human-readable name
`enabled`	boolean	Whether provider is active in routing
`priority`	integer	Base priority for selection
`config`	JSONB	Provider-specific configuration (non-secret)
`capabilities`	JSONB	Supported features, languages, models
`max_concurrency`	integer	Platform concurrency limit
`timeout_ms`	integer	Per-request timeout
`created_at`	timestamptz	Audit
`updated_at`	timestamptz	Audit

`provider_pricing` columns:

Column	Type	Description
`id`	UUID	PK
`provider_id`	text	FK to provider_registry
`operation`	text	`transcribe`, `synthesize`, `complete`, etc.
`unit_type`	text	`seconds`, `characters`, `input_tokens`, `output_tokens`
`unit_cost_usd`	numeric	Current cost per unit
`effective_from`	timestamptz	Rate effective date
`effective_until`	timestamptz	NULL if current
`created_at`	timestamptz	Audit

4.3 Indexing Strategy

Indexing principles:

- Every tenant-scoped table has a composite index starting with `tenant_id`
- High-cardinality columns used in WHERE clauses get dedicated indexes
- Time-series queries use `(tenant_id, created_at DESC)` indexes
- Unique constraints enforce data integrity at the database level
- Partial indexes for status-filtered queries (e.g., `WHERE status = 'active'`)

Critical indexes for query performance:

Table	Index	Query Pattern
`twilio_numbers`	`(phone_number_e164)` UNIQUE	Tenant resolution on every call
`tenant_active_config`	`(tenant_id)` PK	Config version lookup on every call
`call_sessions`	`(tenant_id, started_at DESC)`	Call history listing
`call_sessions`	`(tenant_id, status)`	Active call filtering
`call_events`	`(tenant_id, call_session_id, created_at)`	Event timeline per call
`call_costs`	`(tenant_id, created_at DESC)`	Cost reporting
`call_cost_line_items`	`(tenant_id, call_session_id, created_at)`	Detailed cost breakdown
`audit_log`	`(tenant_id, created_at DESC)`	Audit trail queries
`audit_log`	`(tenant_id, entity_type, entity_id)`	Entity-specific audit queries

4.4 Partitioning Strategy

Tables that require partitioning (at scale):

Table	Partition Key	Partition Scheme	Trigger

`call_sessions`	`started_at`	Monthly range partitions	>10M rows
`call_events`	`created_at`	Monthly range partitions	>50M rows
`call_costs`	`created_at`	Monthly range partitions	>10M rows
`call_cost_line_items`	`created_at`	Monthly range partitions	>100M rows
`call_transcripts`	`created_at`	Monthly range partitions	>10M rows
`audit_log`	`created_at`	Monthly range partitions	>50M rows

Partition management:

- Automated partition creation job runs monthly (creates partition 2 months ahead)
- Old partitions (>24 months) are detached and archived to cold storage
- Partition pruning verified in query EXPLAIN plans for all hot queries
- Partition-aware backup strategy (recent partitions: daily, old: weekly)

4.5 High-Volume Table Strategy

For tables exceeding 100M rows annually:

1. **Write optimization:** Batch inserts for `call_events` and `call_cost_line_items` (buffer in Redis, flush every 5 seconds or at call completion)
2. **Read optimization:** Materialized views for analytics aggregations, refreshed on schedule
3. **Archival:** Move data older than retention period to compressed archive tables or object storage
4. **Retention policy:** Hot data: 90 days in primary table. Warm: 12 months in archive table. Cold: 24+ months in object storage (queryable via external tables)
5. **Vacuum strategy:** Aggressive autovacuum settings on high-write tables (`autovacuum_vacuum_scale_factor = 0.01`)

4.6 Migration Strategy

Migration tooling: Drizzle Kit (`drizzle-kit generate` and `drizzle-kit migrate`)

Migration workflow:

1. Schema change → modify Drizzle schema definitions
2. Generate migration: `drizzle-kit generate`
3. Review generated SQL migration file
4. Test migration on ephemeral database (CI pipeline)
5. Apply to staging environment
6. Validate with integration tests
7. Apply to production (during maintenance window for DDL, online for DML)
8. Verify via automated schema comparison tool

Migration safety rules:

- No destructive migrations without explicit approval (column drops, type changes)
- All migrations are idempotent and reversible
- Data migrations run separately from schema migrations
- Large table migrations use online DDL tools (pg_repack or similar) for zero-downtime
- Migration lock timeout: 5 seconds (fail fast, don't block queries)

---

5. Voice Call Runtime Lifecycle

5.1 Production Lifecycle — Step by Step

Stage 1: Incoming Call Reception

Aspect	Detail
Trigger	Caller dials clinic's dedicated Twilio number
Action	Twilio sends HTTP POST to `/webhooks/twilio/voice/incoming`
Data received	`Called` (E.164), `CallSid`, `From`, `CallStatus`, `Direction`
Latency target	Webhook processed in <100ms
Failure mode	If webhook times out (15s Twilio limit), Twilio plays fallback TwiML

Stage 2: Tenant Resolution

Aspect	Detail
Action	Lookup `twilio_numbers` by `phone_number_e164 → tenant_id`
Cache	Redis cache hit first (5-minute TTL), DB fallback on miss
Validation	Confirm tenant status is `active`
Output	`TenantContext` injected into request
Latency target	<10ms (cache hit), <50ms (DB lookup)
Failure mode	No mapping → safe fallback TwiML + critical alert

Stage 3: Configuration Loading

Aspect	Detail
Action	Load active `config_version` from `tenant_active_config`, hydrate full config snapshot
Cache	Redis cache with 5-minute TTL, keyed by `tenant:{id}:config:{version}`
Validation	Verify config completeness and deployment readiness
Output	Complete `TenantConfigSnapshot` pinned for this session
Latency target	<20ms (cache hit), <100ms (DB hydration)
Failure mode	Config invalid → controlled escalation to human, emit config incident

Stage 4: Session Initialization

Aspect	Detail

Action	Create `call_sessions` record, initialize Redis session state
Data stored	`tenant_id`, `config_version`, `telephony_call_id`, `started_at`, `status: started`
Correlation	Generate unique `correlation_id` for end-to-end tracing
Output	`call_session_id` for all subsequent operations
Latency target	<30ms

Stage 5: Initial Response (Greeting)

Aspect	Detail
Action	Generate greeting based on tenant voice profile and tone settings
TTS	Synthesize greeting audio via selected TTS provider
Delivery	Return TwiML with `<Connect><Stream>` for bidirectional media OR `<Say>`/`<Play>` for simple flows
Content	"Thank you for calling {clinic_name}. How can I help you today?" (styled per tenant tone)
Latency target	<500ms from call answer to audio playback start

Stage 6: Caller Speech Capture (STT)

Aspect	Detail
Action	Stream caller audio to selected STT provider
Provider	Chosen by Provider Selection Engine (cost + latency + reliability score)
Configuration	8kHz mulaw (telephony), language code, interim results enabled
Output	Transcribed text with confidence score
Latency target	<300ms from end-of-speech to final transcription
Failure mode	STT timeout → retry with fallback provider, then "Could you repeat that?"

Stage 7: AI Reasoning

Aspect	Detail
Action	Send runtime context + transcription to Master AI Core via selected LLM provider
Provider	Chosen by Provider Selection Engine
Context	System prompt + tenant config + turn history + current input
Processing	Intent classification → workflow selection → tool calls → policy validation → response generation
Output	Structured turn result: response text, tool call results, updated session state

Latency target	<800ms for reasoning (time-to-first-token <300ms for streaming)
Failure mode	LLM timeout → retry with fallback provider/model, then escalate to human

Stage 8: Tool Execution (Conditional)

Aspect	Detail
Trigger	AI reasoning produces tool call(s)
Action	Execute tool calls in tenant scope (booking, calendar, FAQ lookup)
Isolation	All tool calls include `tenant_id`, integration credentials resolved per-tenant
Output	Tool results fed back to AI for response generation
Latency target	<2s per tool call, <3s for booking confirmation round-trip
Failure mode	Tool timeout → inform caller, offer alternative, log failure

Stage 9: Response Synthesis (TTS)

Aspect	Detail
Action	Convert AI response text to audio using selected TTS provider
Provider	Chosen by Provider Selection Engine
Configuration	Tenant `voice_id`, `speaking_speed`, `tone`, output format (8kHz mulaw)
Optimization	Streaming synthesis — begin playback as chunks arrive
Latency target	<400ms from text to first audio chunk
Failure mode	TTS timeout → fallback provider, then text-to-speech via Twilio `<Say>`

Stage 10: Audio Playback

Aspect	Detail
Action	Stream synthesized audio to caller via Twilio media stream
Delivery	Low-latency streaming to minimize perceived delay
Barge-in	Detect caller interruption and stop playback
Latency target	<50ms stream delivery latency

Stage 11: Cost Attribution

Aspect	Detail
Trigger	Each provider operation during the call
Action	Buffer cost line items in Redis during call, flush to database at call completion

Data captured	Provider ID, operation type, units consumed, unit cost, latency
Aggregation	`call_costs` summary record computed from line items
Latency target	Async — no impact on call latency

Stage 12: Conversation Loop

Aspect	Detail
Cycle	Stages 6–11 repeat for each conversation turn
Context window	Turn history maintained in Redis, windowed to last 10 turns
Loop termination	Caller hangs up, AI determines conversation complete, escalation triggered, or max turns reached
Max turns	Configurable per tenant (default: 30 turns per call)

Stage 13: Session Completion

Aspect	Detail
Trigger	Conversation loop terminates
Actions	1) Update `call_sessions` with final status and summary; 2) Flush all buffered cost line items; 3) Compute and write `call_costs` aggregate; 4) Emit analytics events (`call.completed`); 5) Expire Redis session state
Latency target	<500ms for all completion writes (async from caller perspective)

5.2 Latency Budget Summary

Stage	Target	Budget
Webhook processing	<100ms	100ms
Tenant resolution	<10ms (cached)	10ms
Config loading	<20ms (cached)	20ms
Session init	<30ms	30ms
STT transcription	<300ms	300ms
AI reasoning	<800ms	800ms
Tool execution	<2000ms (when needed)	(parallel with reasoning where possible)
TTS synthesis	<400ms	400ms
Total turn latency (no tool call)	**<1500ms**	**1530ms budget**
Total turn latency (with tool call)	**<3000ms**	**includes tool round-trip**

6. Security Architecture

6.1 Tenant Isolation Enforcement

Isolation boundaries enforced at every layer:

Layer	Isolation Mechanism
API Layer	JWT `tenant_id` claim validated on every request. Tenant A's JWT cannot access Tenant B's data.
Database Layer	Every query includes `WHERE tenant_id = ?`. Row-level security policies as defense-in-depth.
Cache Layer	All Redis keys namespaced: `tenant:{tenant_id}:*`. No shared cache keys for tenant data.
Queue Layer	Queue jobs include `tenant_id` in payload. Worker validates tenant context before processing.
AI Layer	Runtime context envelope is tenant-scoped. System prompt includes only resolved tenant's data.
Integration Layer	Integration credentials resolved per-tenant via secret reference. No cross-tenant credential access.
Logging Layer	All structured logs include `tenant_id`. Log queries are tenant-scoped by default.

Cross-tenant access prevention:

- No API endpoint accepts a raw `tenant_id` parameter from non-admin users
- Tenant ID is always derived from the authenticated session
- Database ORM helpers automatically inject `tenant_id` predicate
- Integration test suite includes cross-tenant access attempt tests (must all fail)

6.2 Row-Level Security

PostgreSQL RLS implementation:

- RLS policies defined on all tenant-scoped tables
- Policy: `USING (tenant_id = current_setting('app.current_tenant_id')::uuid)`
- Session variable `app.current_tenant_id` set by middleware before each request
- RLS enabled as defense-in-depth (application-level filtering is primary)
- RLS cannot be bypassed by application queries (enforced at DB level)

RLS verification:

- Automated test: connect as application user without setting session variable — all tenant queries must return 0 rows
- Automated test: set session variable to Tenant A, attempt to read Tenant B data — must return 0 rows
- Quarterly RLS audit: verify all tenant-scoped tables have active policies

6.3 API Authentication

Authentication flow:

```
Client → POST /auth/login (email + password)
       ← { accessToken (15min), refreshToken (7d, httpOnly cookie) }

Client → GET /api/* (Authorization: Bearer <accessToken>)
       ← Protected resource

Client → POST /auth/refresh (refreshToken cookie)
       ← { accessToken (15min) }
```

Token security:

- Access tokens: JWT, RS256 signed, 15-minute expiry, contains `userId`, `tenantId`, `role`
- Refresh tokens: opaque, stored hashed in database, 7-day expiry, single-use with rotation
- Password hashing: Argon2id with recommended parameters
- Brute-force protection: 5 failed attempts → 15-minute lockout per account

Webhook authentication:

- Twilio webhooks: signature validation using `X-Twilio-Signature` header
- Integration callbacks: HMAC signature validation per integration provider
- Platform-to-platform: mutual TLS or API key with IP allowlist

6.4 Secrets Management

Secret categories and storage:

Secret Type	Storage	Rotation Period	Access Control
Provider API keys (OpenAI, Deepgram, etc.)	AWS Secrets Manager / Vault	90 days	Platform ops only
Twilio credentials	AWS Secrets Manager / Vault	90 days	Telephony service only
JWT signing keys	AWS Secrets Manager / Vault	180 days (with overlap period)	Auth service only
Database credentials	AWS Secrets Manager / Vault	30 days (automated)	Application services only
Integration credentials (per-tenant)	AWS Secrets Manager / Vault	Per integration policy	Integration service + tenant-scoped
Encryption keys	AWS KMS	Annual rotation with key versioning	Encryption service only

Secret access rules:

- No secrets in environment variables in production (fetched from secret manager at startup)
- No secrets in source code, configuration files, or logs

- Secrets are cached in memory with TTL matching rotation period
- Secret access is audited (who read what, when)

6.5 Audit Logging

`audit_log` table:

Column	Type	Description
`id`	UUID	PK
`tenant_id`	UUID	Tenant scope (NULL for platform operations)
`actor_id`	UUID	User or service that performed the action
`actor_type`	enum	`user`, `admin`, `system`, `integration`
`action`	text	Action identifier (e.g., `config.published`, `call.escalated`)
`entity_type`	text	Affected entity type (e.g., `tenant_config`, `call_session`)
`entity_id`	UUID	Affected entity ID
`before_state`	JSONB	State before change (for mutations)
`after_state`	JSONB	State after change (for mutations)
`metadata`	JSONB	Additional context (IP, user agent, correlation ID)
`created_at`	timestamptz	Event timestamp

Audited operations:

- All configuration changes (create, update, publish, rollback)
- All authentication events (login, logout, failed attempt, token refresh)
- All booking actions (confirmed, cancelled, rescheduled)
- All escalation events
- All integration credential changes
- All user role changes
- All platform admin operations

6.6 Rate Limiting

Rate limit tiers:

Scope	Endpoint Pattern	Limit	Window
Per-tenant API	`/api/*`	1000 requests	1 minute
Per-user auth	`/auth/login`	5 requests	15 minutes
Per-tenant webhook	`/webhooks/*`	200 requests	1 minute
Per-tenant config writes	`/api/config/*` (POST/PUT)	30 requests	1 minute
Per-tenant analytics reads	`/api/analytics/*`	60 requests	1 minute

Platform admin	`/admin/*`	300 requests	1 minute

Implementation:

- Sliding window counters in Redis
- `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset` response headers
- HTTP 429 response with `Retry-After` header when limit exceeded
- Rate limit events logged for abuse detection

6.7 Abuse Prevention

Threat	Detection	Prevention
Toll fraud (spoofed inbound calls)	Abnormal call volume from single source	Per-number call rate limiting, geographic origin analysis
AI abuse (adversarial prompts)	Pattern detection in transcriptions	Input guardrails, response validation, prompt injection prevention
Data exfiltration	Anomalous data access patterns	Rate limiting, query monitoring, data access audit
Account takeover	Impossible travel, credential stuffing	Brute-force protection, MFA (future), session invalidation
API abuse	High-frequency automated requests	Progressive rate limiting, CAPTCHA escalation, IP blocking
Cost abuse	Single tenant generating extreme cost	Per-tenant daily/monthly cost caps with auto-suspend

---

7. Scalability Model

7.1 Stateless Workers

Stateless design principles applied to all runtime services:

Service	State Model	Session Data Location
API Gateway	Stateless (JWT auth)	None
Tenant Config Service	Stateless (cache-backed)	Redis cache
Call Routing Service	Stateless	Redis session store
AI Orchestration Service	Stateless	Context injected per request
Provider Selection Engine	Stateless (telemetry cache)	Redis health state
Voice Execution Engine	Stateless	Audio streams are transient
Cost Attribution Service	Stateless (queue-driven)	Redis buffer → PostgreSQL

Implication: Any worker instance can handle any tenant's request. No sticky sessions, no worker-tenant affinity, no local state dependencies.

7.2 Horizontal Scaling Model

Scaling dimensions:

Dimension	Scaling Unit	Metric	Scaling Action
API throughput	API gateway instances	Requests per second	Add instances behind load balancer
Concurrent calls	AI orchestration workers	Active call sessions	Autoscale by session count
Provider throughput	Voice execution workers	Active STT/TTS streams	Autoscale by stream count
Integration load	Integration workers	Pending integration calls	Autoscale by queue depth
Event processing	Analytics workers	Event queue depth	Autoscale by queue backlog
Database connections	Connection pool + read replicas	Connection utilization	Add read replicas, increase pool

Scaling targets by fleet size:

Fleet Size	Concurrent Calls (Peak)	API Gateway Instances	AI Workers	DB Connections
100 clinics	~20	2	4	50
1,000 clinics	~200	4	16	200
5,000 clinics	~1,000	8	64	500
10,000 clinics	~2,000	16	128	1000 (pooled)

7.3 Queue-Based Processing

Queue architecture (BullMQ on Redis):

Queue	Purpose	Concurrency	Priority
`cost-attribution`	Process call cost line items and aggregates	10 workers	Normal
`analytics-events`	Ingest and process analytics events	20 workers	Normal
`integration-callbacks`	Process async integration responses	10 workers	High
`notification-delivery`	Send SMS/email notifications	5 workers	Normal
`config-validation`	Validate configuration changes	3 workers	Low

`recording-processing`	Process and store call recordings	5 workers	Low
`daily-aggregation`	Compute daily analytics summaries	1 worker	Low (scheduled)

Queue reliability:

- Dead-letter queues for all queues (failed jobs moved after 3 retries)
- Exponential backoff on retry (1s, 5s, 30s)
- Job deduplication by idempotency key
- Queue depth monitoring with alerting thresholds
- Graceful shutdown: drain in-progress jobs before termination

7.4 Autoscaling Strategy

Kubernetes HPA (Horizontal Pod Autoscaler) configuration:

Service	Scale Metric	Scale-Up Threshold	Scale-Down Threshold	Min/Max Pods
API Gateway	CPU utilization	70%	30%	2 / 20
AI Workers	Custom metric: active sessions	80% capacity	20% capacity	2 / 150
Voice Workers	Custom metric: active streams	75% capacity	20% capacity	2 / 100
Integration Workers	Queue depth	>50 pending jobs	<5 pending jobs	1 / 20
Analytics Workers	Queue depth	>100 pending events	<10 pending events	1 / 10

Scaling response time:

- Scale-up: <60 seconds (pods pre-pulled, fast startup)
- Scale-down: 5-minute cooldown to prevent thrashing
- Predictive scaling: pre-scale based on historical call patterns (e.g., 9 AM clinic opening)

7.5 Multi-Region Strategy (Future)

Phase 1 (Current): Single region

- All services deployed in one AWS/GCP region
- PostgreSQL with read replicas in same region
- <100ms intra-region latency between services

Phase 2 (Future): Active-passive multi-region

- Primary region handles all writes
- Secondary region has read replicas and standby services
- Failover: promote secondary region, update DNS (target: <5 minute RTO)
- Justification: required when SLA demands >99.95% availability

Phase 3 (Future): Active-active multi-region

- Regional call routing based on caller geography
- CockroachDB or similar for multi-region consistent writes
- Justification: required when regulatory or latency requirements demand data locality

7.6 Handling 10,000+ Clinics

System capacity model at 10,000 tenants:

Metric	Estimate	Design Capacity
Active tenants	10,000	Database and cache designed for 50,000
Concurrent calls (peak)	2,000	Infrastructure sized for 5,000
Calls per day	~100,000	Storage and query design for 500,000/day
Call events per day	~2,000,000	Partitioned tables, stream processing
Total call records (1 year)	~36,000,000	Partitioned with archival
Configuration snapshots	~50,000 (5 versions × 10K)	Standard table
Real-time cache entries	~100,000	Redis cluster with 10GB+

Design decisions enabling 10K+ scale:

- Tenant-partitioned tables prevent single-table hotspots
- Connection pooling (PgBouncer) limits DB connection count
- Redis cluster for cache and session state
- Batch writes for high-volume event tables
- Materialized views for analytics (avoid expensive real-time queries)
- CDN for static dashboard assets
- Pre-computed tenant config snapshots (no JOIN-heavy config loading at call time)

---

8. Observability & Reliability

8.1 Logging Structure

Log format: Structured JSON (pino logger)

Required fields in every log entry:

Field	Description
`timestamp`	ISO 8601 with microseconds
`level`	`trace`, `debug`, `info`, `warn`, `error`, `fatal`
`service`	Service name (e.g., `ai-orchestration`, `telephony`)
`correlationId`	End-to-end request correlation ID
`tenantId`	Tenant ID (null for platform operations)

`callSessionId`	Call session ID (when in call context)
`message`	Human-readable message
`data`	Structured payload (operation-specific)
`error`	Error object with stack trace (when applicable)

Log level guidelines:

Level	Use Case
`trace`	Provider selection scoring details, cache hit/miss
`debug`	Request/response payloads, query parameters
`info`	Call started/completed, booking confirmed, config published
`warn`	Provider degraded, retry triggered, validation warning
`error`	Provider failure, integration error, unexpected exception
`fatal`	Service startup failure, database connection loss

Log destinations:

- Development: stdout (pretty-printed)
- Staging/Production: structured JSON → log aggregator (Datadog, Grafana Loki, or CloudWatch)
- Sensitive data: PII and PHI are never logged. Audio content is never logged. Caller phone numbers are redacted in logs.

8.2 Distributed Tracing

Implementation: OpenTelemetry SDK with automatic instrumentation

Trace propagation:

- W3C TraceContext headers between services
- `correlation_id` propagated through queues and async jobs
- Twilio `CallSid` linked to platform `correlation_id` for end-to-end telephony tracing

Key trace spans:

Span	Parent	Service
`http.incoming_call`	Root	API Gateway
`tenant.resolve`	`http.incoming_call`	Tenant Middleware
`config.load`	`tenant.resolve`	Config Service
`session.create`	`http.incoming_call`	Session Service
`stt.transcribe`	`conversation.turn`	Voice Engine
`ai.reason`	`conversation.turn`	AI Orchestration

`llm.complete`	`ai.reason`	Provider Abstraction
`tool.execute`	`ai.reason`	AI Orchestration
`integration.call`	`tool.execute`	Integration Service
`tts.synthesize`	`conversation.turn`	Voice Engine
`cost.attribute`	`session.complete`	Cost Service

8.3 Metrics to Track

Business metrics:

Metric	Type	Labels
`calls_total`	Counter	`tenant_id`, `outcome`
`calls_active`	Gauge	`tenant_id`
`bookings_total`	Counter	`tenant_id`, `service_category`, `outcome`
`escalations_total`	Counter	`tenant_id`, `reason`
`booking_conversion_rate`	Gauge	`tenant_id`
`ai_resolution_rate`	Gauge	`tenant_id`

System metrics:

Metric	Type	Labels
`http_request_duration_seconds`	Histogram	`method`, `path`, `status`
`provider_request_duration_seconds`	Histogram	`provider_id`, `provider_type`, `operation`
`provider_error_total`	Counter	`provider_id`, `provider_type`, `error_type`
`queue_depth`	Gauge	`queue_name`
`queue_processing_duration_seconds`	Histogram	`queue_name`
`db_query_duration_seconds`	Histogram	`query_type`, `table`
`cache_hit_ratio`	Gauge	`cache_name`
`active_sessions`	Gauge	—
`active_db_connections`	Gauge	`pool_name`

Cost metrics:

Metric	Type	Labels
`call_cost_usd`	Histogram	`tenant_id`, `provider_type`

`provider_spend_total_usd`	Counter	`provider_id`, `operation`
`tenant_monthly_spend_usd`	Gauge	`tenant_id`
`platform_margin_usd`	Counter	`tenant_id`

8.4 Alerting Strategy

Alert severity levels:

Severity	Response Time	Notification Channel	Example
P1 — Critical	<5 minutes	PagerDuty page + Slack + email	All LLM providers down, database unreachable
P2 — High	<30 minutes	PagerDuty + Slack	Primary provider degraded, elevated error rate
P3 — Medium	<4 hours	Slack channel	Single provider failover, increased latency
P4 — Low	Next business day	Email digest	Cost anomaly, queue backlog growing

Critical alert definitions:

Alert	Condition	Action
All providers down (by type)	0 healthy STT/TTS/LLM providers	Immediate page, activate emergency TwiML fallback
Database connection pool exhausted	Available connections = 0 for >30s	Page, investigate connection leak
Call success rate drop	Success rate <90% over 5-minute window	Page, investigate top error
Twilio webhook failure rate	>5% webhook processing errors	Page, check application health
Tenant configuration missing	Active tenant receives call with no valid config	Alert, auto-escalate call to fallback

8.5 SLA Targets

Metric	Target	Measurement
Platform availability	99.9% (8.76 hours downtime/year)	Uptime of call handling pipeline
Call answer rate	99.5%	Calls successfully connected to AI / total calls
Median turn latency	<1.5 seconds	P50 of STT + AI reasoning + TTS per turn

P95 turn latency	<3.0 seconds	P95 of turn latency
Booking confirmation latency	<3.0 seconds	Time from caller confirmation to booking API response
Event pipeline completeness	99.9%	Analytics events successfully ingested / total emitted
Cost attribution accuracy	100%	Every completed call has a cost record
Configuration deployment latency	<30 seconds	Time from publish to runtime availability

8.6 Circuit Breaker Design

Circuit breaker implementation per external dependency:

```
States: CLOSED ──(failures exceed threshold)──► OPEN
                                                  │
        CLOSED ◄──(probe succeeds)── HALF_OPEN ◄──┘
                                        (after recovery timeout)
```

Per-dependency configuration:

Dependency	Failure Threshold	Recovery Timeout	Probe Strategy
LLM providers	5 failures in 60s	30 seconds	Single inference with test prompt
STT providers	5 failures in 60s	30 seconds	Single transcription with test audio
TTS providers	5 failures in 60s	30 seconds	Single synthesis with test text
PMS integrations	3 failures in 60s	60 seconds	Health check endpoint
Calendar integrations	3 failures in 60s	60 seconds	Health check endpoint
Twilio API	5 failures in 60s	30 seconds	Account status API

Circuit breaker observability:

- State transitions emitted as metrics and log events
- Dashboard widget showing all circuit breaker states
- Alert on any circuit breaker opening (P2)

---

9. Cost Optimization Strategy

9.1 Centralized Provider Accounts

Cost advantage of centralization:

Approach	Account Count	Billing Relationships	Volume Tier

Per-tenant accounts	10,000+	10,000+	Individual (highest rates)
Centralized accounts	**3–5**	**3–5**	**Aggregate (lowest rates)**

Savings model (illustrative):

Provider	Individual Rate	Volume Rate (10K+ tenants)	Savings
OpenAI GPT-4o	$5.00/M input tokens	$3.75/M (Tier 4+)	~25%
Deepgram	$0.0059/s	$0.0043/s (Growth plan)	~27%
ElevenLabs	$0.30/1K chars	$0.18/1K chars (Scale)	~40%
Twilio Voice	$0.014/min	$0.010/min (volume commit)	~29%

Estimated blended cost per call (3-minute average):

- STT: ~$0.008 (180s × $0.0043/s ÷ 10 turns, partial seconds)
- LLM: ~$0.015 (avg 2K tokens/call × blended rate)
- TTS: ~$0.009 (avg 500 chars response × 10 turns × rate)
- Telephony: ~$0.030 (3 min × $0.010)
- **Total platform cost: ~$0.062/call**
- **Billed to tenant (30% margin): ~$0.081/call**

9.2 Dynamic Routing

Provider selection optimizations:

Strategy	Mechanism	Impact
Cost-first selection	Select cheapest qualified provider by default	Minimizes base cost
Complexity-based model routing	Use lightweight model (GPT-4o-mini) for FAQ/simple intents, full model (GPT-4o) for booking/complex	40–60% LLM cost reduction
Cache deterministic responses	Cache FAQ answers and clinic hours responses	Eliminate redundant LLM calls
Batch telephony operations	Aggregate Twilio API calls where possible	Reduce per-API-call overhead

Model routing rules:

Intent Category	Complexity	Model Tier	Approximate Token Cost
Greeting, farewell, hold	Low	GPT-4o-mini / Claude Haiku	$0.15/M
FAQ resolution	Low–Medium	GPT-4o-mini / Claude Haiku	$0.15/M
Booking workflow	Medium–High	GPT-4o / Claude Sonnet	$2.50–5.00/M
Complex policy reasoning	High	GPT-4o / Claude Sonnet	$5.00/M

Escalation decision	High	GPT-4o / Claude Sonnet	$5.00/M

9.3 Blended Cost Analysis

Monthly platform economics model (1,000 tenants):

Cost Category	Monthly Estimate	Per Tenant	Per Call
LLM inference	$4,500	$4.50	$0.015
STT transcription	$2,400	$2.40	$0.008
TTS synthesis	$2,700	$2.70	$0.009
Telephony (Twilio)	$9,000	$9.00	$0.030
Infrastructure (compute, DB, Redis)	$3,500	$3.50	$0.012
Total COGS	**$22,100**	**$22.10**	**$0.074**
Revenue (30% margin)	**$28,730**	**$28.73**	**$0.096**
Gross margin	**$6,630**	**$6.63**	**$0.022**

Assumptions: 10 calls/day/tenant average, 3 min/call average, 300,000 calls/month

9.4 Volume Pricing Leverage

Provider negotiation triggers:

Provider	Threshold	Action	Expected Rate Reduction
OpenAI	>$10K/month spend	Negotiate enterprise agreement	15–25%
Deepgram	>10M audio minutes/month	Growth/Enterprise plan	20–35%
ElevenLabs	>100M characters/month	Scale/Enterprise plan	30–45%
Twilio	>$20K/month spend	Volume commit agreement	20–30%

Negotiation cadence: Quarterly provider cost review, annual contract renegotiation

9.5 Budget Alerting

Alert thresholds:

Alert	Condition	Recipient	Action
Tenant daily cost spike	Daily cost > 3× 7-day rolling average	Tenant admin + platform ops	Email notification, investigate

Tenant approaching monthly cap	Monthly cost > 80% of plan limit	Tenant admin	Warning notification
Tenant exceeds monthly cap	Monthly cost > 100% of plan limit	Tenant admin + platform ops	Throttle or suspend per policy
Platform daily provider spend spike	Daily spend > 2× baseline	Platform ops	Investigate, check for abuse
Single call cost anomaly	Call cost > $5.00	Platform ops	Review call, check for loops

9.6 Cost Anomaly Detection

Anomaly detection methods:

Method	Application	Sensitivity
Rolling average deviation	Per-tenant daily cost vs 7-day average	>2× standard deviations
Absolute threshold	Single call cost	>$5.00
Rate-of-change detection	Provider cost rate increase	>10% increase in rolling window
Volume anomaly	Calls per tenant per hour	>3× historical average for that hour
Token count anomaly	Tokens per call	>10,000 tokens (indicates loops)

Anomaly response automation:

1. Flag anomalous call/tenant in monitoring dashboard
2. Emit alert to platform ops channel
3. If cost exceeds hard cap: auto-suspend tenant with notification
4. Platform ops reviews within alert SLA
5. Root cause documented and defenses updated

---

10. DevOps & Deployment Plan

10.1 Environments

Environment	Purpose	Data	Infra Tier	Access
Local dev	Individual development	Seed data, local Docker	Docker Compose (Postgres, Redis)	Developer
CI	Automated testing	Ephemeral per pipeline	Containerized, in-pipeline DB	Automation only
Staging	Pre-production validation	Anonymized production snapshot	Production-like (smaller scale)	Engineering + QA

Production	Live system	Real tenant data	Full production infrastructure	Operations + on-call

Environment parity rules:

- Same Docker images deployed across staging and production
- Same database schema (migrations applied in order)
- Same environment variable structure (values differ)
- Feature flags control feature availability per environment

10.2 CI/CD Pipeline

Pipeline stages:

```
┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────────┐   ┌──────────┐
│  Commit  │──►│  Build   │──►│  Test    │──►│ Deploy to    │──►│ Deploy   │
│  (PR)    │   │  + Lint  │   │  Suite   │   │ Staging      │   │ to Prod  │
└──────────┘   └──────────┘   └──────────┘   └──────────────┘   └──────────┘
                    │              │               │                  │
               TypeScript      Unit tests      Smoke tests       Canary →
               compilation     Integration     E2E tests         Full rollout
               ESLint          Schema diff     Manual QA gate    Health checks
               Prettier        Migration test  Performance test
               Docker build    Security scan
```

CI tooling: GitHub Actions

Pipeline rules:

- All PRs require passing CI before merge
- Code coverage gate: >80% line coverage (no decrease allowed)
- Security scanning: Snyk/Dependabot for dependency vulnerabilities
- Docker image tagged with git commit SHA
- Staging deploy: automatic on merge to `main`
- Production deploy: manual trigger with approval gate

10.3 Database Migration Pipeline

Migration deployment flow:

1. Developer modifies Drizzle schema
2. `drizzle-kit generate` produces migration SQL
3. Migration reviewed in PR
4. CI runs migration on ephemeral DB + validates schema
5. Staging: migration applied automatically on deploy
6. Production: migration applied as pre-deploy step
7. Application startup verifies schema version matches expected

Safety nets:

- Lock timeout on all DDL statements (5s)
- Large table alterations use online DDL tools
- Data-only migrations run as separate queue jobs
- Rollback migration prepared for every forward migration

- Database backup taken before production migration (automated)

10.4 Secrets Rotation

Rotation schedule:

Secret Type	Rotation Period	Method	Downtime
Provider API keys	90 days	Generate new key, update in secret manager, deploy, revoke old	Zero (dual-key overlap)
Database passwords	30 days	Automated rotation via secret manager integration	Zero (connection pool refresh)
JWT signing keys	180 days	Add new key to keyring, sign with new key, old key valid for token lifetime	Zero (keyring supports multiple keys)
Twilio auth token	90 days	Rotate in Twilio console, update secret manager, deploy	Zero (overlap window)
Encryption keys	Annually	KMS key rotation with version, re-encrypt only when accessed	Zero (versioned keys)

10.5 Rollback Strategy

Rollback tiers:

Tier	Scope	Trigger	Method	Target RTO
Application rollback	Application code	Error rate spike, health check failure	Redeploy previous Docker image tag	<5 minutes
Feature rollback	Specific feature	Feature-specific regression	Disable feature flag	<1 minute
Database rollback	Schema change	Migration failure	Apply reverse migration	<15 minutes
Configuration rollback	Tenant config	Bad config deployed	Activate prior config version (`tenant_active_config`)	<30 seconds
Full environment rollback	Everything	Catastrophic failure	Restore from last known-good state	<30 minutes

10.6 Blue/Green and Canary Deployment

Deployment strategy: Canary with progressive rollout

```
Deploy new version to canary pool (5% of traffic)
    │
    ├── Monitor for 10 minutes
```

```
    │      ├── Error rate stable? ──► Proceed
    │      └── Error rate increased? ──► Auto-rollback canary
    │
Increase to 25% of traffic
    │
    ├── Monitor for 10 minutes
    │      ├── Metrics stable? ──► Proceed
    │      └── Degradation? ──► Auto-rollback to 5% or 0%
    │
Increase to 50% of traffic
    │
    ├── Monitor for 15 minutes
    │      ├── All clear? ──► Full rollout
    │      └── Issues? ──► Rollback
    │
Full rollout (100%)
    │
    └── Old version retained for 1 hour (instant rollback capability)
```

Canary health signals:

- HTTP error rate (must be ≤ baseline + 0.5%)
- P95 latency (must be ≤ baseline + 20%)
- Call success rate (must be ≥ baseline - 1%)
- Provider error rate (must be ≤ baseline + 1%)

---

11. Testing Strategy

11.1 Unit Testing

Framework: Vitest (aligned with existing project tooling)

Unit test scope:

Module	Test Focus	Coverage Target
Tenant middleware	Tenant resolution, context injection, authorization	95%
Provider selection	Scoring algorithm, qualification gates, failover logic	95%
Booking logic	Rule validation, slot eligibility, conflict detection	95%
Policy engine	Escalation triggers, safety guardrails, output validation	95%
Cost calculation	Line item computation, aggregation, margin calculation	100%
Configuration validation	Schema validation, cross-field rules, completeness scoring	95%
Prompt construction	Token budget management, context assembly, ordering	90%

Unit test patterns:

- Pure function testing for business logic (no mocks needed)

- Dependency injection for service-layer unit tests
- Test data factories for consistent tenant configuration generation
- Property-based testing for cost calculations and scoring algorithms
- Snapshot testing for prompt construction output

11.2 Integration Testing

Framework: Vitest + Testcontainers (PostgreSQL, Redis)

Integration test scope:

Integration	Test Cases	Environment
Database queries	CRUD operations, tenant isolation, migration verification	Testcontainers PostgreSQL
Redis operations	Cache get/set, session management, rate limiting	Testcontainers Redis
Queue processing	Job creation, processing, retry, dead-letter	Testcontainers Redis (BullMQ)
API endpoints	Full request/response cycle with auth, tenant context	Hono test client
Webhook handling	Twilio signature validation, tenant resolution, session creation	Hono test client
Configuration lifecycle	Create → validate → publish → load at runtime	Full stack

Cross-tenant isolation tests:

- Create two tenants with different configurations
- Verify every query returns only the requesting tenant's data
- Verify cache keys are isolated
- Verify queue jobs are processed with correct tenant context
- Verify RLS policies block cross-tenant reads (database-level)

11.3 Voice Simulation Testing

Purpose: Validate the end-to-end call handling pipeline without real phone calls.

Simulation framework components:

- **Audio generator:** Pre-recorded or TTS-generated caller audio samples
- **Twilio mock:** Simulates Twilio webhook delivery and media stream
- **Integration mock:** Simulates PMS/calendar slot availability and booking responses
- **Scenario runner:** Executes conversational scenarios from YAML test definitions

Test scenario categories:

Category	Scenario Count	Example
Happy path booking	10+	New patient books cleaning, returning patient reschedules
Edge case booking	15+	No available slots, double-booking attempt, too far in future

FAQ resolution	20+	Insurance questions, hours, parking, preparation instructions
Escalation paths	10+	Emergency detection, explicit human request, low confidence
Error handling	10+	Integration timeout, provider failure, invalid config
Tone compliance	5+	Verify response tone matches tenant configuration

Scenario definition format (YAML):

```yaml
scenario: new_patient_booking_happy_path
tenant_config: standard_dental_clinic
turns:
  - caller: "Hi, I'd like to book a cleaning appointment"
    expected_intent: booking.request
    expected_actions: [lookup_available_slots]
  - caller: "Tuesday at 2pm works great"
    expected_intent: booking.confirm_slot
    expected_actions: [create_appointment]
  - caller: "My name is Jane Smith, 555-0123"
    expected_intent: booking.provide_details
    expected_outcome: booking.confirmed
assertions:
  - booking_created: true
  - escalation_triggered: false
  - turn_count: ≤ 6
  - total_latency: ≤ 10000ms
```

11.4 Load Testing

Framework: k6 (JavaScript-based load testing)

Load test scenarios:

Scenario	Concurrent Users	Duration	Success Criteria
Baseline	50 concurrent calls	10 minutes	P95 < 3s, error rate < 1%
Normal load	200 concurrent calls	30 minutes	P95 < 3s, error rate < 1%
Peak load	500 concurrent calls	15 minutes	P95 < 5s, error rate < 2%
Stress test	1000 concurrent calls	10 minutes	Graceful degradation, no crashes
Soak test	200 concurrent calls	4 hours	No memory leaks, stable latency
Spike test	50 → 500 → 50 in 5 min	15 minutes	Recovery within 2 minutes

Metrics captured during load tests:

- Request throughput (RPM)
- Response latency (P50, P95, P99)
- Error rate by endpoint and error type

- Database connection pool utilization
- Redis memory usage and hit rate
- Worker CPU and memory utilization
- Queue depth and processing rate
- Provider response times

11.5 Chaos Testing (Provider Failure Simulation)

Framework: Custom chaos injection middleware + provider mock failure modes

Chaos test scenarios:

Scenario	Injection Method	Expected Behavior
Primary LLM provider down	Return 500 for OpenAI requests	Failover to Anthropic/Google within 2s
All LLM providers degraded	Add 5s latency to all LLM requests	Play hold message, escalate after timeout
STT provider intermittent	30% random failure rate	Retry with fallback, "could you repeat that?"
TTS provider down	Return 500 for ElevenLabs	Failover to Google TTS, slightly different voice
Database connection spike	Exhaust 80% of pool	Graceful queueing, no dropped calls
Redis failure	Kill Redis connection	Fallback to DB reads (degraded performance, no data loss)
PMS integration down	timeout all PMS calls	Offer callback, log failure, inform caller
Network partition	Block specific provider IPs	Circuit breaker opens, failover activates

Chaos testing schedule:

- Automated: weekly in staging environment
- Manual: monthly in production-like isolated environment
- Pre-release: before every major version deployment

---

12. Implementation Timeline

12.1 Milestones

Milestone	Phase	Target Date	Deliverables
M1: Foundation	Phase 1	Week 3	Repository, DB, middleware, base models
M2: Telephony	Phase 2	Week 5	Twilio integration, tenant resolution, call sessions

M3: Provider Layer	Phase 3	Week 8	Provider abstraction, selection engine, health monitoring
M4: AI Engine	Phase 4	Week 12	Master AI Core, prompt orchestration, booking, guardrails
M5: Billing	Phase 5	Week 14	Cost tracking, usage analytics, margin tracking
M6: Dashboard	Phase 6	Week 18	Onboarding flow, voice config, analytics, RBAC
M7: MVP Launch	Integration	Week 20	End-to-end system, 10 pilot clinics
M8: Hardening	Hardening	Week 26	Load testing, chaos testing, security audit, SLA

12.2 Order of Execution

```
Week  1-3:  ################  Phase 1: Core Infrastructure
Week  4-5:  ############      Phase 2: Telephony Integration
Week  4-8:  ########################  Phase 3: Provider Abstraction Layer
Week  9-12:  ######################    Phase 4: AI Conversation Engine
Week 13-14:  ############      Phase 5: Cost Tracking & Billing
Week 11-18:  ##################################  Phase 6: Dashboard
Week 19-20:  ############      Integration testing + MVP launch prep
Week 21-26:  ##############################  Production hardening
```

12.3 Dependencies

```
Phase 1 (Core Infra)
  ├──► Phase 2 (Telephony) ──────────────────────────────┐
  ├──► Phase 3 (Provider Abstraction) ────────────────────┐│
  │     ├──► Phase 4 (AI Engine) ◄── requires L4 interfaces  │
  │     │     ├──► Phase 5 (Cost Tracking) ◄── requires call data│
  │     │     └──► Phase 6 (Dashboard - analytics) ◄── AI data  │
  │     └──► Phase 6 (Dashboard - voice config) ◄── TTS preview │
  └──► Phase 6 (Dashboard - onboarding) ◄── base models only    │
                                                               │
MVP Launch ◄── Phase 2 + Phase 3 + Phase 4 + Phase 5 ◄──────────┘
Production Hardening ◄── MVP Launch + Phase 6 complete ◄──────────┘
```

Critical path: Phase 1 → Phase 3 → Phase 4 → Phase 5 → MVP Launch

Parallelizable work:

- Phase 2 (Telephony) and Phase 3 (Provider Abstraction) can start concurrently after Phase 1
- Phase 6 (Dashboard - onboarding) can start early (depends only on Phase 1 base models)
- Phase 6 (Dashboard - analytics) starts after Phase 4 data is flowing
- Security hardening runs in parallel with Phase 6

12.4 MVP vs Production Hardening

MVP Scope (Week 20)

Component	MVP Capability	Limitation
Tenants	10 pilot clinics	Manual onboarding
Telephony	Inbound call handling	Single Twilio account
Providers	OpenAI (LLM) + Deepgram (STT) + ElevenLabs (TTS)	1 provider per modality, no failover
AI	Booking + FAQ + Escalation	Limited booking integrations
Cost tracking	Per-call cost records	Manual billing reconciliation
Dashboard	Onboarding + basic analytics	Limited chart types
Monitoring	Structured logging + basic alerts	No distributed tracing
Security	JWT auth + tenant isolation	No RLS, basic rate limiting

Production Hardening Scope (Week 21–26)

Component	Hardening Work
Provider resilience	Add fallback providers for each modality, implement full selection engine, build circuit breakers
Load testing	Run all load test scenarios, optimize bottlenecks, verify autoscaling
Chaos testing	Run provider failure simulations, validate degradation paths
Security audit	Penetration testing, RLS implementation, secrets rotation, HIPAA compliance review
Observability	OpenTelemetry integration, Grafana dashboards, PagerDuty alerting, SLA monitoring
Performance optimization	Query optimization, cache tuning, prompt token optimization, streaming TTS
Documentation	API docs, runbooks, incident response playbooks, architecture decision records
Operational readiness	On-call rotation, monitoring dashboards, deployment playbooks, backup verification

---

13. Appendices

Appendix A: Technology Stack Summary

Layer	Technology	Purpose
Frontend	Next.js 15, React 19, Tailwind CSS, shadcn/ui	Dashboard
API	Hono (TypeScript)	API server
Database	PostgreSQL 16, Drizzle ORM	Primary data store

Cache	Redis 7	Sessions, cache, rate limits, queues
Queues	BullMQ	Async job processing
Telephony	Twilio Programmable Voice	Inbound/outbound calls
LLM	OpenAI, Anthropic, Google	AI reasoning
STT	Deepgram, Google Speech, AssemblyAI	Speech transcription
TTS	ElevenLabs, Google TTS, Amazon Polly	Voice synthesis
Observability	Pino, OpenTelemetry, Prometheus, Grafana	Logging, tracing, metrics
CI/CD	GitHub Actions, Docker	Build and deploy
Infrastructure	Docker, Kubernetes (future)	Container orchestration
Secrets	AWS Secrets Manager / HashiCorp Vault	Credentials management
Monorepo	pnpm workspaces, Turborepo	Build orchestration

Appendix B: Glossary

Term	Definition
Tenant	A dental clinic registered on the platform, identified by `tenant_id`
Master AI Core	The shared AI reasoning engine that serves all tenants
Provider Selection Engine	Global service that dynamically routes STT/TTS/LLM requests to optimal providers
Config Version	An immutable snapshot of a tenant's complete configuration
Call Session	A single inbound phone call from start to completion
Turn	One cycle of caller speech → AI reasoning → AI response
Escalation	Handoff from AI to human staff based on policy triggers
Circuit Breaker	A resilience pattern that prevents cascading failures from unhealthy dependencies
RLS	Row-Level Security — PostgreSQL-enforced tenant data isolation
PMS	Practice Management System — clinic software for scheduling and patient records

Appendix C: Risk Register

Risk	Probability	Impact	Mitigation
LLM provider rate limiting under load	Medium	High	Multi-provider abstraction, request queuing, volume agreements

Twilio outage during business hours	Low	Critical	Secondary telephony provider evaluation, voicemail capture fallback
HIPAA compliance gap	Medium	Critical	Early legal review, encryption at rest/transit, audit logging, BAA with providers
Cost per call exceeds margin	Medium	High	Dynamic model routing, aggressive caching, continuous cost monitoring
Prompt injection attacks	Medium	Medium	Input sanitization, output validation, guardrail system, monitoring
Integration partner API changes	Medium	Medium	Versioned adapter interfaces, integration health monitoring, fallback modes
Tenant configuration errors cause bad AI behavior	High	Medium	Validation gates, canary deployment for config changes, easy rollback

---

End of Technical Build Plan

Document version 1.0.0 — March 3, 2026

This document is the authoritative technical reference for the AI Receptionist Generation Platform. All implementation work should conform to the architecture, interfaces, and constraints defined herein.